

COM4532 — Blockchain Technologies

Project Proposal

**Predicting Ethereum Gas Prices Using Deep Learning:
An LSTM-Based Time Series Forecasting Approach**

Due Date: May 4, 2025 — 11:59 PM

Boran Ömer Doğan-21290594

Arif Erdem Orhan- 21290337

Furkan Çağatay Özbek-21290216

1. Introduction

1.1. Background and Problem Statement

Ethereum is designed as a global, open-source blockchain platform for running decentralized applications (dApps) and smart contracts [1]. Every computation performed on the network is metered using a unit called "gas," which serves a dual purpose: it ensures fair allocation of computational resources and protects the network from spam attacks [1]. Users pay these gas fees in Ether (ETH), the native cryptocurrency of the Ethereum network. However, Ethereum's throughput is fundamentally constrained by a limited number of transactions per second, creating a persistent bottleneck between network capacity and user demand. This imbalance produces extreme volatility in gas prices, making cost prediction a genuinely difficult problem.

For everyday users and decentralized application developers, this volatility translates directly into financial uncertainty. During periods of peak demand — such as a high-profile NFT launch or acute instability in decentralized finance (DeFi) markets — gas fees can spike by an order of magnitude within minutes. Users who set gas prices too low risk having their transactions sit unconfirmed for hours; those who set them too high end up paying far more than necessary. A reliable gas price prediction model would allow both groups to make more informed decisions and significantly reduce wasted expenditure.

1.2. The Evolution of Gas Fees and EIP-1559

The gas fee mechanism underwent a fundamental redesign with Ethereum Improvement Proposal 1559 (EIP-1559), which was activated as part of the London Hard Fork in August 2021. Before this update, gas pricing operated as a first-price auction: users competed for block space by bidding higher and higher fees, with miners naturally prioritizing the most profitable transactions [2]. The result was a highly opaque and unpredictable system in which users had no reliable anchor for setting a reasonable price.

EIP-1559 replaced the auction with a two-component fee structure. The first component is a protocol-determined Base Fee that is algorithmically adjusted block by block based on how full the previous block was. The Base Fee is burned rather than paid to validators, removing it from circulation permanently. The second component is a voluntary Priority Fee (tip) paid directly to validators to incentivize faster inclusion [2]. Because the

Base Fee adjusts predictably — rising by up to 12.5% when a block exceeds its gas target and falling by the same proportion when it falls short — fee changes became considerably smoother from one block to the next. Nevertheless, EIP-1559 does not eliminate large-scale volatility caused by sudden surges in network demand, and accurately anticipating those surges remains an open challenge [2].

The update rule governing the Base Fee can be expressed formally as follows:

$$\text{BaseFee}_{t+1} = \text{BaseFee}_t \times \left(1 + 0.125 \times \frac{\text{GasUsed}_t - \text{GasTarget}_t}{\text{GasTarget}_t} \right) \quad \text{E}$$

equation 1: EIP-1559 Base Fee update rule

Where: BaseFee_{t+1} is the base fee for the next block; GasUsed_t is the gas consumed in the current block; $\text{GasTarget} = \text{GasLimit} / 2$ (the 50% utilization target); the maximum per-block change is $\pm 12.5\%$.

1.3. Limitations of Current Solutions

Wallet applications and blockchain explorers — such as MetaMask and Etherscan — do provide users with gas fee estimates before they submit transactions. However, academic research consistently shows that these tools carry significant limitations [4]. Most current estimators rely on simple heuristics: a weighted or unweighted average of recent block prices, or a straightforward oracle that reports the minimum gas price accepted in recent blocks [4]. While these reactive approaches capture the current network state reasonably well, they fail to anticipate future price movements. They cannot predict the sharp spike that will arrive in ten minutes when a popular NFT collection opens for minting, nor can they tell a patient user that waiting an hour will reduce their fees by 70%. What is needed is a proactive and dynamic forecasting system rather than a reactive snapshot [4].

1.4. The Need for a Data-Driven Approach

Blockchain networks are uniquely well-suited to data-driven analysis. Every transaction, every block, and every fee paid is recorded permanently and publicly on the chain. This transparency provides access to a high-quality historical record that would be impossible to obtain for most financial systems. Crucially, the relationships between on-chain metrics — such as network congestion, pending transaction counts, and base fee changes — are highly non-linear, and the price signal itself is noisy and non-stationary. These properties

make classical statistical approaches such as simple moving averages ill-equipped to capture the full dynamics of the system [3].

Deep learning architectures, and Long Short-Term Memory (LSTM) networks in particular, are well-suited to this kind of problem. LSTM models are specifically designed to learn long-range dependencies in sequential data by maintaining a cell state that can propagate information across many time steps without the vanishing-gradient problem that plagues vanilla recurrent networks [3]. This makes them effective at capturing the slow seasonal rhythms in gas prices — such as the well-documented pattern of lower fees on weekends and overnight — as well as the fast, event-driven spikes that characterize periods of high demand.

1.5. Dataset and Project Scope

This project adopts a data-driven approach in which historical on-chain network metrics are used as inputs to forecast the optimal gas price one day ahead. The dataset is assembled from Etherscan’s publicly available chart exports, covering the post-EIP-1559 period from August 5, 2021 to December 31, 2023 — approximately 880 daily observations. This date range was chosen deliberately: by restricting the analysis to the period after the London Hard Fork, the dataset reflects a consistent fee mechanism throughout, avoiding the structural break that would arise from mixing pre- and post-EIP-1559 data.

The dataset comprises six daily time series: average gas price (Gwei), total gas used, average gas limit, transaction count, ETH price in USD, and a derived network utilization ratio (gas used divided by gas limit). These features collectively capture both the supply side of block space and the demand side of transaction activity, providing the model with sufficient context to distinguish routine fluctuations from genuine demand shocks.

2. Problem Formulation

2.1. Task Definition

This project is formulated as a multivariate time series regression problem. Given a sequence of historical network metrics, the objective is to predict the expected gas price — expressed in Gwei — for the following day. Unlike classification approaches that bin prices into low/medium/high categories, the regression formulation produces a continuous numerical prediction that can be directly used by a user or application to set an informed gas limit.

The choice of a regression framework is motivated by the practical use case: a user or smart contract scheduler does not simply need to know whether tomorrow’s fees will be “high” or “low” — they need a concrete Gwei estimate to plan transactions accordingly. The problem is solved using Long Short-Term Memory (LSTM) networks, which are well-established for capturing non-linear dependencies and long-term patterns in time series data [3], supplemented by an XGBoost model that serves as an interpretable machine learning baseline.

2.2. Data Description

The model relies exclusively on on-chain data derived from the transparent, publicly accessible structure of the Ethereum blockchain. All features are collected at daily granularity from Etherscan’s public chart exports and cover the period from August 5, 2021 — the activation date of EIP-1559 — through December 31, 2023.

The primary entity of interest is the Ethereum block: each block validated by the network’s validators carries a base fee determined by EIP-1559 and records how much of its available gas capacity was actually consumed [2]. At the aggregate daily level, this yields metrics such as average gas price, total gas used, and average gas limit. Transactions are the second key entity; each transfer or smart contract execution contributes to the overall demand for block space, and the daily transaction count provides a direct measure of that demand [1]. Finally, the ETH/USD price is included as a proxy for broader market sentiment: periods of rapid price appreciation tend to be accompanied by increased on-chain activity and higher gas fees.

2.3. Input / Output Definition

The model takes a sliding window of the past $w = 30$ days as input and produces a single-step prediction for day $t + 1$. Formally, the input tensor at time t is defined as follows:

$$X_t = \begin{bmatrix} x_{t-w} \\ x_{t-w+1} \\ \vdots \\ x_{t-1} \end{bmatrix} \in \mathbb{R}^{w \times d}, \quad \hat{y}_{t+1} \in \mathbb{R}$$

Equation 2: Input tensor definition (sliding window)

Where: $X_t \in \mathbb{R}^{(w \times d)}$ is the input matrix; $w = 30$ is the lookback window size (days); $d = 6$ is the number of input features; $x_{t-i} \in \mathbb{R}^d$ denotes the feature vector at time step $t - i$; the model output is $\hat{y}_{t+1} \in \mathbb{R}$, the predicted gas price in Gwei for the following day.

The six input features are: (1) average gas price in Gwei, (2) total daily gas used, (3) average block gas limit, (4) network utilization ratio (gas used / gas limit), (5) daily transaction count, and (6) ETH price in USD. All features are normalized to the $[0, 1]$ range using min-max scaling prior to model training. The target variable is the average gas price on day $t + 1$, derived by shifting the gas price series forward by one step.

2.4. Challenges and Constraints

Extreme Volatility and Outliers

In a regression setting, the analogue of class imbalance is the presence of extreme outliers. Gas prices on the Ethereum network are typically stable at a moderate level, but they can increase by an order of magnitude within a single day during acute demand events such as major NFT drops or DeFi liquidation cascades [4]. The model must learn to treat these spikes as genuine signals rather than noise, which requires careful consideration of loss functions and normalization strategies. Standard mean-squared error loss, for instance, penalizes large errors quadratically and may cause the model to over-fit on spike events at the expense of routine-period accuracy.

Non-Stationarity

Gas price time series are inherently non-stationary: the mean and variance of the series shift substantially across the observation window, partly as a function of broader market cycles and partly due to structural changes such as network upgrades. Standard train-test splitting strategies that assume stationarity are therefore inappropriate. This project

addresses non-stationarity through chronological splitting and walk-forward validation, ensuring that the model is always evaluated on data that is strictly later in time than the training period.

Label Definition

Even with a clean dataset, defining the ground truth gas price for a given day is not entirely straightforward. The average gas price aggregates transactions with very different urgency levels, from routine token transfers to time-critical arbitrage bots [4]. Using the daily average as the prediction target is a pragmatic choice that aligns with the estimates provided by existing oracles, but it does smooth over intra-day variation. This limitation is acknowledged as a boundary condition of the current project scope.

3. Proposed Method / Approach

3.1. Feature Extraction

Rather than working directly with raw transaction-level data — which would require processing millions of records per day — the project uses aggregated daily statistics that capture the essential dynamics of the network at a manageable scale. The six features described in Section 2.3 were selected because they collectively represent the two sides of the gas price equilibrium: supply (gas limit, utilization ratio) and demand (transaction count, gas used, ETH price as a market sentiment proxy).

An additional engineered feature is the day-over-day change in base fee, computed as the first difference of the gas price series. This delta captures the directional momentum of fees and gives the model information about whether the network is currently in a tightening or loosening phase. Temporal features encoding the day of the week are also considered, as gas prices exhibit a well-documented weekly seasonality with lower fees on weekends [4].

3.2. Planned Models

Long Short-Term Memory (LSTM) Network

The primary model is a two-layer LSTM network implemented in TensorFlow/Keras. LSTM networks address the vanishing gradient problem inherent in standard recurrent networks through a gating mechanism that allows the model to selectively retain or discard information across long sequences [3]. The forget gate, input gate, and output gate together control what information persists in the cell state from one time step to the next. The update equations governing a single LSTM cell are as follows:

$$\begin{aligned}f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \\i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \\\tilde{C}_t &= \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \\C_t &= f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \\o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \\h_t &= o_t \odot \tanh(C_t)\end{aligned}$$

Equation 3: LSTM gate equations

Where: f_t = forget gate; i_t = input gate; o_t = output gate; $\tilde{C}_t$ = candidate cell state; C_t = updated cell state; h_t = hidden state output; σ = sigmoid activation; $\odot$ = element-wise (Hadamard) product; W and b are learned weight matrices and bias vectors.

The network architecture consists of a first LSTM layer with 64 hidden units that returns the full sequence, followed by a Dropout layer with rate 0.2 to regularize against over-fitting. A second LSTM layer with 32 hidden units then reduces the sequence to a single context vector, followed by a second Dropout layer. A Dense layer with 16 units and ReLU activation provides a non-linear projection, and a final single-unit Dense layer produces the scalar gas price prediction. The model is trained using the Adam optimizer with a mean-squared error loss function, and training is stopped early when validation loss fails to improve for 10 consecutive epochs.

XGBoost (Gradient Boosted Trees)

An XGBoost model is implemented as a strong machine learning baseline and as a complementary tool for feature importance analysis. Unlike LSTM, XGBoost operates on a flat feature representation rather than a sequence, so the 30-day input window is flattened into a 180-dimensional feature vector (30 timesteps $\times$ 6 features). XGBoost's built-in feature importance scores will be used to identify which historical time steps and which individual metrics are most informative for the prediction task, providing an interpretable complement to the LSTM's learned representations.

3.3. Explainability with SHAP

Deep learning models are frequently criticized as black boxes: they may achieve strong predictive performance while remaining opaque about why they made a particular prediction. To address this, SHAP (SHapley Additive exPlanations) values will be computed for the LSTM model using the DeepExplainer interface from the shap library. SHAP values assign each input feature a contribution score for a given prediction, grounded in cooperative game theory. This will allow the project to answer questions such as: when the model predicts an unusually high gas price, is that prediction driven primarily by the current transaction count, by a recent surge in ETH price, or by the base fee trajectory over the past week? These explanations add practical value beyond the raw prediction and strengthen the scientific contribution of the project.

3.4. High-Level Pipeline

The end-to-end pipeline proceeds through four stages. In the data collection stage, five daily time series are downloaded as CSV files from Etherscan’s public chart exports (average gas price, total gas used, average gas limit, transaction count, and ETH price). These are merged on date, filtered to the post-EIP-1559 period, and a utilization ratio feature is computed. In the preprocessing and feature engineering stage, missing values are handled by forward-filling (typically fewer than five missing days in the full dataset), outliers beyond the 99th percentile are Winsorized rather than removed to preserve the signal of genuine spike events, and all features are normalized using min-max scaling fit exclusively on the training set to avoid data leakage.

In the modeling stage, the processed data is split chronologically into training (80%), validation (10%), and test (10%) sets, and a 30-day sliding window is applied to generate input-output pairs. The LSTM model is trained on the training set with early stopping monitored on the validation set, and hyperparameters are tuned through a lightweight grid search over learning rate and batch size. In the evaluation stage, the model’s predictions on the held-out test set are compared against both the LSTM predictions and three baselines using MAE, RMSE, MAPE, and R^2 , and SHAP values are computed to explain a sample of the model’s predictions.

4. Evaluation Plan

4.1. Evaluation Metrics

Because this is a regression task that produces continuous numerical predictions, model performance is quantified using four complementary metrics. Each captures a different aspect of prediction quality, and reporting all four together provides a more complete picture than any single metric alone.

$$\begin{aligned} \text{MAE} &= \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \\ \text{RMSE} &= \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \\ \text{MAPE} &= \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \\ R^2 &= 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \end{aligned}$$

Equation 4: Evaluation metrics: MAE, RMSE, MAPE, and R²

Where: y_i = actual gas price for day i (Gwei); $\hat{y}_i$ = predicted gas price; $\bar{y}$ = mean of actual values; n = number of test observations. MAE reports average absolute deviation in Gwei; RMSE penalizes large errors more heavily and is particularly sensitive to spike mispredictions; MAPE expresses error as a percentage and is useful for communicating prediction accuracy to non-specialist audiences; R^2 measures the proportion of variance in the target explained by the model, where 1.0 is a perfect fit.

Among these, RMSE is particularly meaningful in this context because gas price spikes — the most consequential and hardest-to-predict events — produce large absolute errors. A model that performs well on RMSE has learned to handle spikes more gracefully than one that merely minimizes average error. R^2 provides a useful normalizing frame: a model that simply predicts the historical mean at all times achieves an R^2 of 0, so any positive R^2 indicates the model has extracted genuine predictive signal from the features.

4.2. Train / Test Strategy

Time series data violates the independent-and-identically-distributed assumption that underlies standard random cross-validation. Randomly shuffling the data before splitting would allow future information to leak into the training set, producing optimistically biased performance estimates. This project therefore uses a strict chronological split: the earliest 80% of observations (approximately August 2021 to August 2023) form the training set, the next 10% (approximately August to November 2023) serve as the validation set for early stopping and hyperparameter selection, and the final 10% (approximately November to December 2023) constitute the held-out test set on which all reported metrics are computed.

In addition to this single-split evaluation, a walk-forward validation procedure is applied during hyperparameter search. In walk-forward validation, the model is trained on all data up to a given point, used to predict one step ahead, and then re-trained with the new observation included before predicting the next step. This more accurately reflects the real-world deployment scenario in which the model would be retrained periodically as new data becomes available.

4.3. Baseline Comparison

The predictive value of the LSTM model is demonstrated by comparing it against three baseline approaches of increasing sophistication. The first and simplest baseline is the Naïve model, which predicts that tomorrow’s gas price will equal today’s price. This represents the theoretical lower bound of a useful forecasting model: any model that cannot outperform the naïve predictor has failed to extract any temporal structure from the data. The second baseline is a Simple Moving Average (SMA) computed over the previous seven days, which approximates the heuristic approach used by most current gas price oracles, including MetaMask [4]. Outperforming SMA demonstrates that the deep learning model adds value over the industry standard.

The third baseline is an ARIMA (AutoRegressive Integrated Moving Average) model, which represents the gold standard for classical statistical time series forecasting. ARIMA is fitted to the training set using standard order selection procedures (examining ACF/PACF plots and minimizing AIC), providing a rigorous statistical benchmark against which the LSTM’s non-linear modeling capacity can be measured. Outperforming ARIMA, particularly on RMSE during spike periods, would constitute strong evidence that deep learning captures dynamics that classical statistics cannot.

5. References

5.1. Academic Papers

- [1] Buterin, V. (2014). Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform. Ethereum White Paper. Retrieved from <https://ethereum.org/en/whitepaper/>
- [2] Roughgarden, T. (2021). Transaction Fee Mechanism Design for the Ethereum Blockchain: An Economic Analysis of EIP-1559. *ACM SIGecom Exchanges*, 19(1), 52–56. <https://doi.org/10.1145/3484128.3484137>
- [3] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- [4] Werner, S. M., Pritz, P. J., & Wattenhofer, R. (2020). Step on the Gas? A Better Approach for Recommending the Ethereum Gas Price. In *Proceedings of the 2020 Crypto Valley Conference on Blockchain Technology (CVCBT)*. IEEE.
- [5] Mars, R., Abid, A., Cheikhrouhou, S., & Kallel, S. (2021). A Machine Learning Approach for Gas Price Prediction in Ethereum Blockchain. In *Proceedings of the 2021 IEEE 45th Annual Computers, Software, and Applications Conference (COMPSAC)*, pp. 156–165. <https://doi.org/10.1109/COMPSAC51774.2021.00033>
- [6] Butt, T. M., Wei, Z., Afzal, F., & Khan, N. (2023). Blockchain Transaction Fee Forecasting: A Comparison of Machine Learning Methods. *Mathematics*, 11(9), 2212. MDPI. <https://doi.org/10.3390/math11092212>

5.2. Data Sources

- [7] Etherscan (2024). Ethereum Charts and Statistics: Average Gas Price, Gas Used, Gas Limit, Transactions, and Ether Price. Retrieved from <https://etherscan.io/charts>
- [8] Etherscan (2024). Ethereum Gas Tracker. Retrieved from <https://etherscan.io/gastracker>

5.3. Software and Frameworks

- [9] Abadi, M., et al. (2015). TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. Available at <https://tensorflow.org>. Used for building and training the LSTM model.

- [10] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794. <https://doi.org/10.1145/2939672.2939785>